МЕТОДИЧЕСКИЕ УКАЗАНИЯ
к практическому занятию № 1

Тема: «Анализ требований и функциональная декомпозиция: перевод бизнес-требований заказчика в блок-схемы алгоритмов и спецификации функций»

Вариант А — базовая траектория · Вариант Б — продвинутая траектория

Работа рассчитана на 1 пару (2 академических часа) · Выполнение в парах (Системный аналитик + Разработчик)

Параметр	Значение
Дисциплина	Программирование и прототипирование ИТ-решений
Направление подготовки	09.03.03 Прикладная информатика (профиль «Бизнес и системная аналитика»)
Семестр / трудоёмкость	1 семестр, 2 академических часа (1 пара)
Среда разработки	Visual Studio Code (с поддержкой Markdown / Mermaid), Git
Форма контроля	Отчёт о выполнении работы (на пару) + взаимная приёмка (Cross-Check)
Обеспеченность ПО	Python 3.11+, Git, VS Code (подготовлены в ЛР № 1–3)

1. Цель работы

Научиться работать в связке «Аналитик — Разработчик»: принимать «сырые» текстовые бизнес-требования от заказчика (главного технолога месторождения), декомпозировать задачу на логические шаги, строить блок-схемы алгоритмов и оформлять строгие технические спецификации функций на Python (сигнатуры, аннотации типов, docstrings) до написания основного кода.

В результате выполнения работы студенты научатся:

- выделять из неструктурированного рассказа заказчика входные параметры, граничные условия, формулы и бизнес-правила;
- выполнять функциональную декомпозицию — разбивать монолитную задачу (знакомую по ЛР № 2 и ЛР № 3) на независимые, переиспользуемые функции;
- визуализировать логику ветвлений (if-elif-else, match-case) и циклов (while True) в виде блок-схем (по ГОСТ или в нотации Mermaid прямо в VS Code);
- разрабатывать спецификации функций на языке Python: заголовки def, аннотации типов аргументов и возвращаемых значений (float, str, tuple), а также подробные строки документации (docstring) с заглушками pass / ...;
- проверять реализуемость требований путём взаимной верификации между ролями в паре.

2. Обязательное ядро (одинаково для обоих вариантов)

- работа выполняется в паре с явным разделением ролей: Студент 1 — Системный аналитик (CA), Студент 2 — Ведущий разработчик (Dev) (на середине занятия при выполнении упражнений роли меняются);
- из текста бизнес-кейса составлена таблица требований (входы, ограничения, формулы, выходы);

- нарисована блок-схема алгоритма (допускается: диаграмма Mermaid в файле .md, схема в draw.io или аккуратная схема от руки на бумаге с фото в отчёт);
- создан файл specs.py, содержащий не менее 4 спецификаций функций с аннотациями типов (type hints) и полными докстрингами (Args, Returns, Business Rule);
- файл specs.py запускается без синтаксических ошибок (python specs.py), а ОДНА ключевая функция полностью реализована и проверена на индивидуальных данных пары;
- в репозитории Git сделано не менее двух осмысленных коммитов (документация аналитика -> код спецификаций разработчика);
- выполнены ВСЕ 4 обязательных упражнения на понимание (раздел 8) с фиксацией в отчёте.

3. Глоссарий: все термины работы

3.1. Требования и декомпозиция

Термин	Что это значит
Бизнес-требование	Пожелание заказчика на языке бизнеса («хочу видеть, какие скважины нерентабельны»), часто неполное или размытое.
Функциональное требование	Точное описание поведения системы: что программа принимает на вход, как считает и что выдаёт на выход.
Функциональная декомпозиция	Разбиение одной большой программы (как ваш main.py в ЛР № 2–3) на мелкие функции, где каждая отвечает ровно за ОДНУ задачу.
Бизнес-правило (Business Rule)	Условие или формула из предметной области (например: «при обводнённости >= 80% скважина считается водяной»).

3.2. Блок-схемы алгоритмов

Термин	Что это значит
Блок-схема	Графическое представление алгоритма, где шаги показаны фигурами, связанными стрелками.
Терминатор (овал / скруглённый блок)	Начало или конец алгоритма (вход в функцию / возврат return).
Процесс (прямоугольник)	Вычисление по формуле или присваивание (например, q_oil = q_liq * (1 - w)).
Решение / Условие (ромб)	Проверка условия (if-elif-else, while). Из ромба всегда выходят минимум две стрелки: Да (True) и Нет (False).
Ввод / Вывод (параллелограмм)	Получение данных от пользователя (input) или печать на экран (print).
Mermaid	Текстовый язык разметки, позволяющий рисовать блок-схемы прямо внутри README.md в VS Code с помощью кода.

3.3. Спецификация функции в Python

Термин	Что это значит
Спецификация функции	«Контракт» функции: её имя, входные параметры с типами, тип результата и текстовое описание логики без необходимости знать внутренний код.
Сигнатура функции	Строка def имя(параметр: тип) -> тип_возврата:. Определяет интерфейс вызова.
Аннотация типов (Type Hints)	Указание ожидаемых типов данных через двоеточие и стрелку: def calc(q: float) -> float:.
Docstring (докстринг)	Многострочный комментарий в тройных кавычках сразу под def, описывающий назначение, аргументы (Args) и результат (Returns).
Заглушка (Stub / pass / ...)	Пустое тело функции (pass или многоточие ...), позволяющее согласовать архитектуру и запустить файл до написания расчётного кода.

4. Тайминг занятия и распределение ролей в паре (1 пара = 90 минут)

Так как на занятие отведена ровно одна пара (90 минут), работа строится по принципу парного проектирования. Сразу распределите роли:

- **Роль 1: Системный аналитик (СА) —** читает запрос заказчика, выписывает формулы и ограничения в таблицу, рисует блок-схему, диктует граничные условия и проверяет работу разработчика.
- **Роль 2: Ведущий разработчик (Dev) —** создаёт структуру проекта и Git, переводит схему аналитика в сигнатуры Python-функций (specs.py), пишет аннотации типов, docstrings и реализует проверочную функцию.

Рекомендуемый график пары:

- **00–10 мин:** Знакомство с заданием, расчёт общего номера варианта пары N = N1 + N2, создание папки и .venv.
- **10–30 мин (Параллельная фаза):** СА составляет таблицу требований и черновик блок-схемы; Dev настраивает Git и готовит каркас specs.py.
- **30–55 мин (Сборка спецификаций):** Совместный перевод блок-схемы в код спецификаций specs.py и чистовая отрисовка схемы в README.md.
- **55–75 мин:** Выполнение 4 экспресс-упражнений (со сменой ролей!) и реализация одной функции.
- **75–90 мин:** Сверка «прогноз | факт», коммиты в Git и сдача работы преподавателю.

5. Общая подготовка и расчёт варианта пары

Шаг 5.1. Вычислить индивидуальный номер пары N

Пусть N1 — номер по журналу первого студента, а N2 — номер по журналу второго студента. Общий параметр вашей пары вычисляется по формуле: N = N1 + N2. (Если работаете в одиночку — берите свой номер N). Запишите N1, N2 и итоговый N в шапку отчёта.

Шаг 5.2. Создать папку проекта и окружение (выполняет Разработчик)

```
mkdir well-specs-prac1
cd well-specs-prac1
python -m venv .venv
```

```
.\.venv\Scripts\Activate.ps1
```

6. Вариант А (базовый): Декомпозиция и спецификация «Экспресс-оценка рентабельности скважины»

6.1. Бизнес-требование от заказчика (легенда кейса)

К вашей паре обратился ведущий технолог цеха добычи нефти и газа (ЦДНГ) со следующим запросом: «Ребята, в прошлых работах вы написали мне калькулятор и валидатор в одном длинном файле main.py. Сейчас мы хотим встроить этот модуль в общую систему предприятия, но наши программисты не понимают, как устроена ваша логика! Мне нужно, чтобы вы разбили программу на отдельные понятные функции и задокументировали их. Суть задачи простая: оператор вводит дебит жидкости Qж (от 0 до 1000 м³/сут) и обводнённость W (от 0 до 100 %). Если ввели ерунду — переспрашиваем. Дальше считаем дебит чистой нефти Qн (в м³/сут) и переводим его в массовый дебит Мн (в т/сут, плотность нефти принять равной 850 кг/м³). В конце система должна выдать статус рентабельности: если Мн меньше порога M_min — статус 'Убыточна (стоп)', если от M_min до 3*M_min — 'Рентабельна (база)', а если больше или равно 3*M_min — 'Высокодебитная (приоритет)'. Нарисуйте мне понятную схему этого процесса и подготовьте спецификации всех функций!»

6.2. Индивидуальные исходные данные пары (Вариант А)

```
N     = N1 + N2                 (сумма номеров студентов пары по журналу)
Qж    = 50 + 3 * (N % 25)       (контрольный дебит жидкости, м³/сут)
W     = 15 + 2 * (N % 35)       (контрольная обводнённость, %)
M_min = 8.0 + 0.5 * (N % 10)    (порог рентабельности по нефти для вашей пары, т/сут)
```

Шаг 1. Анализ требований и функциональная декомпозиция (ведёт Системный аналитик)

Создайте в папке проекта файл README.md и опишите декомпозицию на 4 атомарные функции:

- **1. read_valid_param(prompt, min_val, max_val) —** циклический запрос и валидация одного числа в границах;
- **2. calc_oil_volume(q_liq, w_pct) —** расчёт объёмного дебита нефти Qн = Qж * (1 - W / 100);
- **3. calc_oil_mass(q_oil, density) —** перевод объёма нефти в тонны Мн = Qн * плотность / 1000;
- **4. evaluate_profitability(m_oil, m_min) —** классификация статуса рентабельности через if-elif-else.

Шаг 2. Построение блок-схемы алгоритма (Системный аналитик + Разработчик)

Добавьте в README.md следующую блок-схему на языке Mermaid (или нарисуйте в draw.io / на бумаге):

```
```mermaid
flowchart TD
    Start([Начало работы]) --> InputQ[/Ввод Qж и W с валидацией границ/]
    InputQ --> CheckValid{Значения в пределах\nQж: 0..1000, W: 0..100?}
    CheckValid -- Нет --> ErrMsg[/Вывод ошибки диапазона/]
    ErrMsg --> InputQ
    CheckValid -- Да --> CalcVol[Расчёт объёма нефти:\nQн = Qж * 1 - W/100]
    CalcVol --> CalcMass[Расчёт массы нефти:\nМн = Qн * 850 / 1000]
    CalcMass --> Cond1{Мн < M_min?}
    Cond1 -- Да --> Status1[Статус: Убыточна - стоп]
```
```

```
    Cond1 -- Нет --> Cond2{Мн < 3 * M_min?}
    Cond2 -- Да --> Status2[Статус: Рентабельна - база]
    Cond2 -- Нет --> Status3[Статус: Высокодебитная - приоритет]
    Status1 --> Output[/Вывод сводки: Qн, Мн, Статус/]
    Status2 --> Output
    Status3 --> Output
    Output --> End([Конец работы])
```

Шаг 3. Разработка спецификаций функций в Python (ведёт Разработчик)

Создайте файл specs.py со спецификациями 4 функций и рабочей реализацией evaluate_profitability:

```
"""Спецификации функций модуля оценки рентабельности скважины (Вариант А).

Практическое занятие № 1. Выполнено в паре.
"""

OIL_DENSITY: float = 850.0  # кг/м³
M_MIN_THRESHOLD: float = 10.5  # ЗАМЕНИТЕ на ваш индивидуальный M_min!


def read_valid_param(prompt: str, min_val: float, max_val: float) -> float:
    """Запрашивает у пользователя число в цикле до попадания в диапазон [min_val; max_val].

    Бизнес-правило (BR-01): Система не должна пропускать отрицательные значения,
    пустые строки и текстовый мусор. Запятая автоматически приводится к точке.

    Args:
        prompt (str): Текст приглашения для оператора.
        min_val (float): Нижняя допустимая граница (включительно).
        max_val (float): Верхняя допустимая граница (включительно).

    Returns:
        float: Проверенное вещественное число в границах [min_val; max_val].
    """
    pass


def calc_oil_volume(q_liq: float, w_pct: float) -> float:
    """Вычисляет суточный объёмный дебит чистой нефти по дебиту жидкости и обводнённости.

    Формула: Q_н = Q_ж * (1 - W / 100).

    Args:
        q_liq (float): Дебит пластовой жидкости, м³/сут (0.0 <= q_liq <= 1000.0).
        w_pct (float): Обводнённость продукции в процентах (0.0 <= w_pct <= 100.0).

    Returns:
        float: Объёмный дебит нефти Q_н, м³/сут.
    """
    ...


def calc_oil_mass(q_oil: float, density: float = OIL_DENSITY) -> float:
    """Переводит объёмный дебит нефти (м³/сут) в массовый дебит (т/сут).
```

```
    Формула: M_н = Q_н * density / 1000.

    Args:
        q_oil (float): Объёмный дебит нефти, м³/сут.
        density (float, optional): Плотность нефти, кг/м³. По умолчанию 850.0.

    Returns:
        float: Массовый дебит нефти M_н, т/сут.
    """
    ...


def evaluate_profitability(m_oil: float, m_min: float = M_MIN_THRESHOLD) -> str:
    """Определяет категорию рентабельности скважины по массовому дебиту нефти.

    Бизнес-правило (BR-02):
    - Если m_oil < m_min -> 'Убыточна (стоп)'
    - Если m_min <= m_oil < 3 * m_min -> 'Рентабельна (база)'
    - Если m_oil >= 3 * m_min -> 'Высокодебитная (приоритет)'

    Args:
        m_oil (float): Массовый дебит нефти, т/сут.
        m_min (float): Минимальный порог рентабельности для участка, т/сут.

    Returns:
        str: Строковый статус рентабельности скважины.
    """
    if m_oil < m_min:
        return "Убыточна (стоп)"
    elif m_oil < 3 * m_min:
        return "Рентабельна (база)"
    else:
        return "Высокодебитная (приоритет)"


if __name__ == "__main__":
    print("ПРОВЕРКА СПЕЦИФИКАЦИЙ МОДУЛЯ (Вариант А)")
    print("=" * 45)
    print(evaluate_profitability.__doc__)
    print("-" * 45)
    test_mass = 25.0  # Подставьте сюда свою расчётную массу M_н!
    status = evaluate_profitability(test_mass)
    print(f"При M_н = {test_mass:.2f} т/сут (порог {M_MIN_THRESHOLD}): -> {status}")
```

Шаг 4. Ручной расчёт (прогноз) и запуск проверки

```
python specs.py
git init
git add README.md
git commit -m "CA: добавлены бизнес-требования и блок-схема алгоритма"
git add specs.py
git commit -m "Dev: разработаны спецификации функций и проверка статуса"
```

7. Вариант Б (продвинутый): Декомпозиция подсистемы «Аварийная диагностика и маршрутизация технологического режима»

7.1. Бизнес-требование от заказчика (легенда кейса)

К вашей паре обратился главный инженер нефтепромысла:
«Коллеги, мы автоматизируем диспетчерскую службу. Нам нужно спроектировать программный модуль диагностики скважин по четырём показателям: дебит жидкости Qж (0..1000 м³/сут), обводнённость W (0..100 %), пластовое давление Рпл (5..60 МПа) и забойное давление Рзаб (1..60 МПа). Логика должна быть строго модульной:
1) Сначала считаем депрессию на пласт dP = Рпл - Рзаб и коэффициент продуктивности J = Qж / dP (внимание: если dP <= 0, делить нельзя — возвращаем 0.0!).
2) Далее система присваивает скважине код технического состояния (str): если dP < 0 — 'BLOWOUT'; иначе если dP <= 0.5 — 'IDLE'; иначе если W >= 85.0 — 'WATERED'; иначе если J < J_norm — 'CLOGGED'; во всех остальных случаях — 'NORMAL'.
3) Наконец, по полученному коду через match-case формируется директива диспетчеру (текст команды и уровень критичности от 0 до 3). Разделите систему на чистые функции, оформите архитектуру в пакете src/ и нарисуйте блок-схему!»

7.2. Индивидуальные исходные данные пары (Вариант Б)

```
N      = N1 + N2                  (сумма номеров студентов пары по журналу)
Qж     = 40 + 4 * (N % 20)        (дебит жидкости, м³/сут)
W      = 20 + 3 * (N % 28)        (обводнённость, %)
Рпл    = 22.0 + 0.5 * (N % 15)    (пластовое давление, МПа)
Рзаб   = 14.0 + 0.4 * (N % 18)    (забойное давление, МПа)
J_norm = 4.5 + 0.25 * (N % 12)    (норматив коэфф. продуктивности для вашей пары,
м³/(сут·МПа))
```

Шаг 1. Блок-схема алгоритма диагностики (Вариант Б)

```
```mermaid
flowchart TD
    Start([Вход: Qж, W, Рпл, Рзаб]) --> CalcDP[Расчёт депрессии:\nΔP = Рпл - Рзаб]
    CalcDP --> CheckDP{ΔP > 0?}
    CheckDP -- Нет --> SetJZero[J = 0.0\nЗащита от деления на 0]
    CheckDP -- Да --> CalcJ[Расчёт продуктивности:\nJ = Qж / ΔP]
    SetJZero --> Diag1{ΔP < 0?}
    CalcJ --> Diag1
    Diag1 -- Да --> CodeBlow[Код: BLOWOUT]
    Diag1 -- Нет --> Diag2{ΔP <= 0.5?}
    Diag2 -- Да --> CodeIdle[Код: IDLE]
    Diag2 -- Нет --> Diag3{W >= 85%?}
    Diag3 -- Да --> CodeWat[Код: WATERED]
    Diag3 -- Нет --> Diag4{J < J_norm?}
    Diag4 -- Да --> CodeClog[Код: CLOGGED]
    Diag4 -- Нет --> CodeNorm[Код: NORMAL]
    CodeBlow --> MatchRoute[[Маршрутизация match-case:\nформирование директивы и уровня
0..3]]
    CodeIdle --> MatchRoute
    CodeWat --> MatchRoute
    CodeClog --> MatchRoute
    CodeNorm --> MatchRoute
```
```

```
    MatchRoute --> End([Возврат кортежа: Директива, Уровень])
```

Шаг 2. Конфигурация и спецификации функций (config.py и specs.py)

Файл src/well_design/config.py:

```
"""Граничные условия и нормативы технологического режима (Вариант Б)."""

Q_LIMITS: tuple[float, float] = (0.0, 1000.0)
W_LIMITS: tuple[float, float] = (0.0, 100.0)
P_RES_LIMITS: tuple[float, float] = (5.0, 60.0)
P_BHP_LIMITS: tuple[float, float] = (1.0, 60.0)

W_CRITICAL: float = 85.0  # Критическая обводнённость, %
DP_MIN_ACTIVE: float = 0.5  # Минимальная рабочая депрессия, МПа
J_NORM_DEFAULT: float = 5.5  # ЗАМЕНИТЕ на ваш индивидуальный J_norm!
```

Файл src/well_design/specs.py:

```
"""Спецификации функций подсистемы аварийной диагностики (Вариант Б)."""

from well_design import config


def validate_telemetry(
    q_liq: float, w_pct: float, p_res: float, p_bhp: float
) -> tuple[bool, str]:
    """Проверяет вхождение всех параметров телеметрии в допустимые физические границы.

    Args:
        q_liq (float): Дебит жидкости, м³/сут.
        w_pct (float): Обводнённость, %.
        p_res (float): Пластовое давление, МПа.
        p_bhp (float): Забойное давление, МПа.

    Returns:
        tuple[bool, str]: Кортеж (Флаг_успеха, Сообщение_о_результате).
    """
    ...


def calc_hydraulics(q_liq: float, p_res: float, p_bhp: float) -> tuple[float, float]:
    """Вычисляет депрессию на пласт (ΔP) и коэффициент продуктивности (J).

    Бизнес-правило (BR-02): ΔP = p_res - p_bhp. Если ΔP <= 0, коэффициент
    продуктивности J принимается равным 0.0 (защита от ZeroDivisionError).

    Args:
        q_liq (float): Дебит жидкости, м³/сут.
        p_res (float): Пластовое давление, МПа.
        p_bhp (float): Забойное давление, МПа.

    Returns:
        tuple[float, float]: Кортеж (drawdown_mpa, productivity_index).
    """
```

```
    ...


def determine_status_code(
    drawdown: float,
    w_pct: float,
    prod_index: float,
    j_norm: float = config.J_NORM_DEFAULT,
) -> str:
    """Классифицирует техническое состояние скважины по приоритетной цепочке условий."""
    if drawdown < 0.0:
        return "BLOWOUT"
    elif drawdown <= config.DP_MIN_ACTIVE:
        return "IDLE"
    elif w_pct >= config.W_CRITICAL:
        return "WATERED"
    elif prod_index < j_norm:
        return "CLOGGED"
    else:
        return "NORMAL"


def route_dispatcher_action(status_code: str) -> tuple[str, int]:
    """Маршрутизирует код состояния в текстовую директиву диспетчеру через match-case."""
    ...


if __name__ == "__main__":
    sample_dp, sample_w, sample_j = 8.0, 35.0, 6.5
    code = determine_status_code(sample_dp, sample_w, sample_j)
    print(f"Вход: ΔP={sample_dp}, W={sample_w}%, J={sample_j} -> Код: {code}")
```

8. Обязательные упражнения на понимание (выполняются со сменой ролей!)

- **Упражнение 1. Инспекция докстрингов через help():** Добавьте в конец specs.py вызов help(имя_функции). Сделайте скриншот вывода в терминале и объясните, откуда Python берёт этот текст.
- **Упражнение 2. Оживление заглушки по спецификации партнёра:** Бывший Аналитик (теперь Разработчик) заменяет заглушку ... на реальный код (в Варианте А — calc_oil_volume и calc_oil_mass; в Варианте Б — calc_hydraulics и route_dispatcher_action), не меняя сигнатуру def. В отчёт приведите таблицу «Прогноз Аналитика | Факт программы».
- **Упражнение 3. Трассировка маршрута на блок-схеме («Сухой прогон»):** Пройдите по стрелкам вашей схемы от «Начало» до «Конец» с вашими числами и запишите маршрут одной строкой в отчёт.
- **Упражнение 4. Изменение бизнес-требований («Внезапная правка заказчика»):** Вариант А: при W == 100% сразу возвращать статус 'Чистая вода (авария)'. Вариант Б: при Рзаб > 50 МПа возвращать код 'OVERPRESSURE'. Внесите правку сначала в блок-схему, затем в docstring и в код.

9. Структура отчёта (один общий отчёт на пару)

1) Титульная часть с ролями (CA и Dev) и расчётом N = N1 + N2; 2) Цель работы; 3) Индивидуальные данные пары и таблица требований; 4) Блок-схема алгоритма; 5) Код specs.py; 6) Таблица «Прогноз | Факт»; 7) Результаты 4 упражнений; 8) Ответы на контрольные вопросы; 9) Вывод.

10. Частые ошибки и их решение

Симптом	Причина и решение
SyntaxError: expected an indented block	После def func(): забыли тело функции. Добавьте хотя бы docstring или заглушку pass / ...
help(func) выводит None	Описание написано через # над функцией, а не в тройных кавычках первой строкой ВНУТРИ функции.
На блок-схеме из ромба выходит одна стрелка	У любого условия (if или while) всегда два исхода — Да (True) и Нет (False). Дорисуйте вторую ветку.
Вся логика написана в одной функции main()	Нарушена декомпозиция: ввод, расчёт и классификация должны быть в разных функциях с return.

11. Контрольные вопросы и эталонные ответы для подготовки к защите

Вопрос 1 (Системному аналитику)

Формулировка: В чём разница между исходным бизнес-требованием технолога и функциональной спецификацией? Покажите на примере вашей работы одну фразу заказчика и то, в какие конкретные ограничения/формулы вы её перевели.

Вопрос 2 (Системному аналитику)

Формулировка: Какими фигурами на блок-схеме обозначаются: а) ввод данных с клавиатуры; б) расчёт по формуле; в) проверка условия if-elif-else? Чем отличается изображение цикла while от обычного ветвления if на блок-схеме?

Вопрос 3 (Разработчику)

Формулировка: Что такое сигнатура функции и аннотации типов (type hints)? Заставляет ли запись q_liq: float сам интерпретатор Python автоматически преобразовывать строку в число, или это инструмент для разработчика и анализаторов кода?

Вопрос 4 (Разработчику)

Формулировка: Почему при декомпозиции задачи мы разделили расчёт физических показателей и определение текстового статуса на отдельные функции с return, вместо того чтобы считать и печатать всё сразу внутри одной процедуры?